

Blue Hawk Ops

Software Requirements Specification (SRS), Software Architecture Document (SAD), and UniFi Integration Plan

Document status: Draft for review

Product name: Blue Hawk Ops

Organization: Blue Hawk Technologies / WDBTECHNOLOGY, S.R.L.

Primary user groups: IT support technicians, network operators, infrastructure administrators, managers, and authorized client viewers

Primary operating location: Santo Domingo, Dominican Republic

Document purpose: Define the purpose, scope, requirements, architecture, security boundaries, data model, phased roadmap, and integration plan for Blue Hawk Ops.

Important operating rule: Blue Hawk Ops must complement—not replace—vendor management platforms. UniFi Network/Site Manager remains the authoritative operational platform for UniFi configuration. FortiGate/FortiManager remains authoritative for firewall configuration. Blue Hawk Ops initially collects, normalizes, documents, reports, and correlates approved information. It must begin read-only and must not make unapproved production changes.

Part I — Product Purpose and Vision

1. Executive Summary

Blue Hawk Ops is an internal operations platform for Blue Hawk Technologies. It centralizes visibility of client sites, network devices, assets, incidents, documentation, health indicators, and approved operational reports.

The product solves a common managed-support problem: important information is distributed across vendor consoles, spreadsheets, messaging applications, technicians' memory, ad-hoc documents, and individual accounts. A technician may know that an access point is offline, but not immediately see its physical location, responsible client, last incident, last firmware review, switch/uplink relationship, asset ownership, or existing troubleshooting runbook.

Blue Hawk Ops provides a controlled internal source of operational context. It integrates vendor data gradually, beginning with approved read-only UniFi information. It correlates that data with the company's inventory, incidents, documentation, and client/site records. FortiGate data may be added later as a separate, higher-risk read-only connector.

The product is not intended to replace UniFi, FortiGate, FortiManager, FortiAnalyzer, a dedicated SIEM, a complete RMM, or a full ITSM platform. Its value is the company-specific layer above those tools: normalization, cross-client organization, asset reconciliation, operational documentation, incident context, reporting, and controlled workflow.

2. Product Vision

Give an authorized Blue Hawk technician a reliable answer to: **What is happening, where is it happening, what is affected, what has already been tried, what can be safely done next, and how do we document the result?**

The platform should make operations more consistent without making them reckless. It should reduce time spent locating information, prevent stale inventory, expose recurring issues, improve client reporting, and create an evidence trail for support and infrastructure decisions.

3. Product Principles

- **Read-only first:** the first integration versions retrieve approved data and never alter network configuration.
- **Human accountability:** the application may recommend or prepare actions; people approve and perform production-impacting changes.
- **Least privilege:** integrations and users receive only the access needed for their role.
- **Native systems remain authoritative:** vendor consoles retain configuration authority; Blue Hawk Ops stores normalized operational views and company workflow data.
- **Data freshness is visible:** every integrated value must show or be traceable to its most recent synchronization time.
- **Tenant isolation:** one client's data must not be visible to another client or to unauthorized staff.
- **Evidence over assumptions:** the platform separates observed facts, imported vendor data, technician notes, automated inferences, and confirmed conclusions.
- **Documentation is operational:** incident notes, change records, runbooks, and asset history are first-class features.
- **Safe automation:** automated actions start with monitoring, reporting, and notifications; changes require explicit controls.
- **Build small, validate early:** release a pilot that solves one real operational problem before expanding scope.

Part II — Scope and Users

4. Problem Statement

Blue Hawk may support internal and client environments that include network gateways, firewalls, switches, access points, servers, cameras, NVRs, workstations, storage, cloud services, and software applications. Device and site information can change frequently. Without a centralized operations layer, the team risks:

- Incomplete or outdated inventory.
- Longer troubleshooting time due to fragmented information.

- Repeated incidents without documented root cause or prevention.
- Unclear equipment ownership, location, warranty, and lifecycle.
- Difficulty creating client-ready operational reports.
- Limited evidence for maintenance, upgrades, changes, and support decisions.
- Unsafe automation or untracked configuration changes.
- Inconsistent access control across clients and technicians.

5. In Scope

The initial product scope includes:

- Organization/client and site management.
- Asset and network-device inventory.
- Physical locations: building, floor, office, room, rack, storage area, and other defined locations.
- UniFi read-only integration for approved sites.
- Device discovery, inventory reconciliation, and discrepancy review.
- Site/device health summaries and data-freshness indicators.
- Incident tracking and linked knowledge-base articles.
- Runbooks and operational documentation.
- Client/site reporting.
- User authentication, role-based access control, audit logging, and integration-run records.
- Controlled export of authorized data.
- A foundational internal REST API for the web dashboard and future approved integrations.

6. Out of Scope for Initial Releases

The following are explicitly not part of the first release:

- Direct modification of UniFi configuration.
- Direct modification of FortiGate firewall, VPN, routing, VLAN, or security policies.
- Automatic firmware upgrades or device restarts.
- A full replacement for UniFi Network, UniFi Site Manager, FortiGate, FortiManager, FortiAnalyzer, Wazuh, or a commercial RMM/ITSM platform.
- A complete security information and event management system.
- Billing, accounting, payroll, customer invoicing, or full CRM.
- Unrestricted remote command execution on endpoints.
- AI that makes or executes infrastructure/security decisions autonomously.
- Exposure of management interfaces or secrets to external/public users.

7. Stakeholders and Personas

Persona	Goals	Typical permissions
Platform administrator	Manage application users, roles, integrations, retention, and security settings	Full application administration; no implied vendor-config write access
Network operator	Understand network health, investigate connectivity, create incidents, review device records	Read site/device data; create/update incidents; request changes
Support technician	Resolve user, endpoint, Wi-Fi, printer, application, and basic network incidents	Read approved operational data; create/update assigned incidents; access runbooks
Infrastructure administrator	Maintain servers, backups, virtualization, storage, and approved integrations	Read infrastructure records; manage approved assets and documentation
Manager	View status, workload, trends, risks, and reports	Read dashboards/reports for authorized organizations
Client viewer	View narrowly scoped reports/incidents for their own organization	Read-only, tenant-restricted access
Auditor	Review access, changes, incidents, evidence, and controls	Read-only access to authorized records/audit logs

Part III — Software Requirements Specification

8. Functional Requirements

The format uses a priority designation:

- **Must:** required for initial product value or safety.
- **Should:** important, planned after the core MVP is stable.
- **Could:** valuable future enhancement.

8.1 Identity, Access, and Tenant Isolation

ID	Priority	Requirement
FR-IAM-01	Must	The system shall authenticate all users before granting access to protected functions or data.
FR-IAM-02	Must	The system shall support at least these roles: platform administrator, network operator, support technician, manager, client viewer, and auditor.
FR-IAM-03	Must	The system shall enforce authorization on the server side for every protected API endpoint.
FR-IAM-04	Must	The system shall restrict client viewers to records belonging only to their authorized organization/site scope.
FR-IAM-05	Must	The system shall record authentication events, failed access attempts, role changes, and privileged administrative actions.

ID	Priority	Requirement
FR-IAM-06	Should	The system shall support MFA for privileged roles.
FR-IAM-07	Should	The system shall integrate with an approved identity provider if one becomes available.

8.2 Organizations, Sites, and Locations

ID	Priority	Requirement
FR-SITE-01	Must	The system shall allow authorized users to create and manage organizations/clients.
FR-SITE-02	Must	The system shall allow each organization to contain one or more sites.
FR-SITE-03	Must	The system shall store site details including name, status, address/description, timezone, primary contacts, and notes.
FR-SITE-04	Must	The system shall support hierarchical physical locations such as building, floor, office, room, rack, warehouse, shelf, and common area.
FR-SITE-05	Should	The system shall provide a site-health summary using the latest available integration and incident data.
FR-SITE-06	Should	The system shall allow authorized users to attach diagrams, documents, and approved reference links to a site.

8.3 Asset and Network Inventory

ID	Priority	Requirement
FR-ASSET-01	Must	The system shall create, view, update, search, filter, and retire asset records.
FR-ASSET-02	Must	An asset record shall support an internal asset code, category, vendor, model, serial number, MAC address where applicable, management IP where applicable, lifecycle status, location, responsible party, notes, and timestamps.
FR-ASSET-03	Must	The system shall support categories including computers, laptops, servers, switches, access points, firewalls, cameras, NVRs, UPS devices, printers, phones, tools, spare parts, and other.
FR-ASSET-04	Must	The system shall track asset statuses including new, active, in storage, assigned, under repair, damaged, lost, retired, and disposed.
FR-ASSET-05	Must	The system shall retain an audit history for material asset changes.
FR-ASSET-06	Should	The system shall support photo and document attachments subject to type, size, authorization, and malware-handling controls.

ID	Priority	Requirement
FR-ASSET-07	Should	The system shall support CSV import/export with validation and a reviewable import result.
FR-ASSET-08	Could	The system shall generate and scan QR/barcode labels for asset lookup and movement updates.

8.4 UniFi Read-Only Integration

ID	Priority	Requirement
FR-UNIFI-01	Must	The system shall support a dedicated UniFi integration credential with least privilege and without exposing the credential to browser clients.
FR-UNIFI-02	Must	The system shall allow authorized administrators to connect a configured UniFi organization/site scope to Blue Hawk Ops.
FR-UNIFI-03	Must	The system shall retrieve approved read-only site and network-device information at a configured interval.
FR-UNIFI-04	Must	The system shall record integration-run start time, end time, success/failure status, error category, and number of records processed.
FR-UNIFI-05	Must	The system shall record data freshness for imported site/device records.
FR-UNIFI-06	Must	The system shall normalize discovered devices into internal records while retaining a stable external identifier and source reference.
FR-UNIFI-07	Must	The system shall support asset reconciliation using a safe matching strategy, prioritizing stable identifiers such as an existing mapped external ID and MAC address.
FR-UNIFI-08	Must	The system shall create reviewable discrepancies when a discovered device is not matched, a known device is no longer observed, or key attributes conflict.
FR-UNIFI-09	Must	The initial UniFi connector shall not execute configuration changes, device restarts, firmware updates, or network-policy modifications.
FR-UNIFI-10	Should	The system shall display approved device details such as device type, name, model, MAC, management IP, firmware, connectivity state, last seen, and client count where available.
FR-UNIFI-11	Should	The system shall provide authorized deep links to the native UniFi console when such links can be safely constructed.
FR-UNIFI-12	Should	The system shall display integration failures and stale-data warnings prominently to authorized operators.

8.5 Incidents, Troubleshooting, and Knowledge Base

ID	Priority	Requirement
FR-INC-01	Must	The system shall allow authorized users to create, assign, update, and close incidents.

ID	Priority	Requirement
FR-INC-02	Must	An incident shall include an ID, organization, site, priority, status, reporter, owner, affected asset/device where known, symptoms, timestamps, diagnostic notes, resolution, and verification result.
FR-INC-03	Must	The system shall support P1, P2, P3, and P4 priorities or an approved equivalent severity model.
FR-INC-04	Must	The system shall allow incidents to be linked to assets, network devices, locations, alerts, change requests, and knowledge-base articles.
FR-INC-05	Must	The system shall retain a chronological activity history for incident actions and status changes.
FR-INC-06	Should	The system shall provide guided troubleshooting templates for recurring categories such as no internet, DNS failure, Wi-Fi issue, offline device, VPN issue, printer issue, slow endpoint, disk full, server unavailable, and application connectivity failure.
FR-INC-07	Should	The system shall allow a resolved incident to be converted into a reviewable knowledge-base article or linked to an existing article.
FR-KB-01	Must	The system shall allow authorized users to create, search, categorize, revise, and archive knowledge-base articles and runbooks.
FR-KB-02	Must	A knowledge-base article shall distinguish symptoms, evidence, diagnosis, resolution, verification, prevention, and escalation criteria.
FR-KB-03	Should	The system shall support review dates, owners, version history, and approval status for operational runbooks.

8.6 Alerts, Health, and Reports

ID	Priority	Requirement
FR-OBS-01	Must	The system shall show a site/device health status based on defined rules and the freshness of imported data.
FR-OBS-02	Must	The system shall avoid creating duplicate alerts for the same ongoing condition.
FR-OBS-03	Must	The system shall require repeated failed observations or another defined confirmation policy before declaring a device offline automatically.
FR-OBS-04	Must	The system shall allow an alert to be linked to an incident and show its lifecycle: opened, acknowledged, resolved, suppressed, or false positive.
FR-OBS-05	Should	The system shall generate scheduled reports for authorized users, including site summary, devices offline, inventory discrepancies, open/closed incidents, data freshness, and maintenance reminders.
FR-OBS-06	Should	The system shall support report export in approved formats such as CSV and PDF/HTML, subject to authorization.
FR-OBS-07	Could	The system shall send approved notifications through email, Teams, Slack, or webhooks.

8.7 Change Management and Future Controlled Actions

ID	Priority	Requirement
FR-CHG-01	Must	The system shall allow authorized users to create change requests with objective, affected site/assets, risk, implementation plan, validation steps, rollback plan, schedule, and approver fields.
FR-CHG-02	Must	The system shall preserve the history of approvals, changes, validation results, and closure status for each change request.
FR-CHG-03	Should	The system shall allow authorized users to attach before/after configuration exports or references, subject to storage and access controls.
FR-CHG-04	Could	A future controlled-action module shall require explicit confirmation, approved maintenance window, RBAC, audit logging, target verification, rate limiting, post-action health validation, and documented rollback before performing an external write action.
FR-CHG-05	Must	No external write action shall be enabled by default.

8.8 FortiGate Read-Only Integration — Future Phase

ID	Priority	Requirement
FR-FGT-01	Should	The system shall support a FortiGate connector using a dedicated least-privilege integration identity.
FR-FGT-02	Should	The initial connector shall retrieve only approved read-only data such as appliance identity, FortiOS version, interface/WAN status, VPN status, configuration-backup status, and selected policy/security summaries.
FR-FGT-03	Should	The system shall protect sensitive policy, topology, event, and security-log details using strict role and tenant controls.
FR-FGT-04	Should	The system shall generate review reminders for firmware, privileged accounts, backup status, policy ownership, temporary rules, and change references.
FR-FGT-05	Must	The initial FortiGate connector shall not create, modify, reorder, disable, or delete policies, objects, VPNs, routing, VLANs, administrators, or firmware configuration.

9. Non-Functional Requirements

9.1 Security

ID	Requirement
NFR-SEC-01	All application traffic shall use HTTPS/TLS in production.
NFR-SEC-02	API credentials, secrets, tokens, private keys, and passwords shall not be stored in source code, browser storage, logs, screenshots, tickets, or unencrypted documentation.

ID	Requirement
NFR-SEC-03	The system shall use server-side secret handling through environment variables protected by deployment controls or an approved secrets manager.
NFR-SEC-04	The system shall enforce least privilege for users, database accounts, background workers, and vendor integration credentials.
NFR-SEC-05	The system shall use strong password hashing for local passwords and shall not store plaintext passwords.
NFR-SEC-06	The system shall validate all external input, including API payloads, import files, query parameters, attachments, and user-submitted fields.
NFR-SEC-07	The system shall protect against common web risks including broken access control, injection, insecure file upload, sensitive-data exposure, CSRF where applicable, and insecure direct object references.
NFR-SEC-08	The system shall create audit events for privileged actions, data exports, role/permission changes, integration configuration changes, and future external write actions.
NFR-SEC-09	Production administration shall be restricted to approved networks, VPN, or equivalent controls where feasible.
NFR-SEC-10	The system shall maintain dependency-update and vulnerability-scanning practices.

9.2 Reliability and Recovery

ID	Requirement
NFR-REL-01	Integration failures shall not corrupt existing inventory or remove records automatically.
NFR-REL-02	The system shall retain the last known imported value and label it stale when synchronization fails or exceeds a defined freshness threshold.
NFR-REL-03	Background jobs shall use timeouts, bounded retries, backoff, and failure logging.
NFR-REL-04	The system shall create scheduled database backups and document restore procedures.
NFR-REL-05	Restore procedures shall be tested at a defined interval.
NFR-REL-06	The system shall make destructive operations explicit and require confirmation.

9.3 Performance and Scalability

ID	Requirement
NFR-PERF-01	Standard dashboard and search views should respond within 3 seconds under expected initial operating load, excluding approved long-running report generation.
NFR-PERF-02	Vendor synchronization shall run asynchronously and shall not block normal dashboard use.

ID	Requirement
NFR-PERF-03	The system shall paginate large asset, device, incident, and audit-log listings.
NFR-PERF-04	The architecture shall allow connector workers and the web/API service to scale independently if the number of sites or telemetry volume grows.
NFR-PERF-05	The system shall define retention and aggregation rules before collecting high-frequency telemetry at scale.

9.4 Usability and Accessibility

ID	Requirement
NFR-UX-01	The web interface shall work on desktop, laptop, tablet, and mobile browsers for the functions appropriate to each device.
NFR-UX-02	The interface shall make data source and freshness understandable; it must not present stale vendor data as real-time truth.
NFR-UX-03	Critical states shall not rely only on color; use labels, icons, and accessible contrast.
NFR-UX-04	The system shall provide concise error messages that help authorized users understand what failed without exposing secrets or sensitive topology.
NFR-UX-05	Mobile workflows shall support field work such as asset lookup, location update, photo capture, and incident-note entry.

9.5 Maintainability and Observability

ID	Requirement
NFR-MNT-01	The codebase shall use clear module boundaries between web UI, internal API, domain logic, persistence, integrations, and background jobs.
NFR-MNT-02	Each vendor connector shall be independently testable and isolated from core domain logic.
NFR-MNT-03	The project shall use version control, code review, automated tests, dependency locking, and documented deployment steps.
NFR-MNT-04	The system shall emit structured logs with correlation IDs for HTTP requests and integration runs.
NFR-MNT-05	The system shall expose service health checks suitable for monitoring.
NFR-MNT-06	API contracts shall be documented through OpenAPI and versioned under an internal API version such as <code>/api/v1</code> .

9.6 Privacy and Data Governance

ID	Requirement
NFR-DATA-01	The system shall collect and retain only data needed for authorized operational purposes.
NFR-DATA-02	The system shall document data categories, sources, owners, access scope, retention periods, and deletion/archival rules.
NFR-DATA-03	The system shall restrict sensitive client topology, security events, credentials, and personal data according to role and tenant scope.
NFR-DATA-04	The system shall support a documented process for correcting inventory errors and preserving relevant audit history.
NFR-DATA-05	Before production use, the organization shall review legal, contractual, privacy, and client-authorization implications of collecting and presenting client data.

Part IV — Software Architecture Document

10. Architectural Goals

Blue Hawk Ops should be simple enough for a small technical team to understand and operate, while disciplined enough to safely handle multi-site and potentially multi-client infrastructure data.

The architecture must:

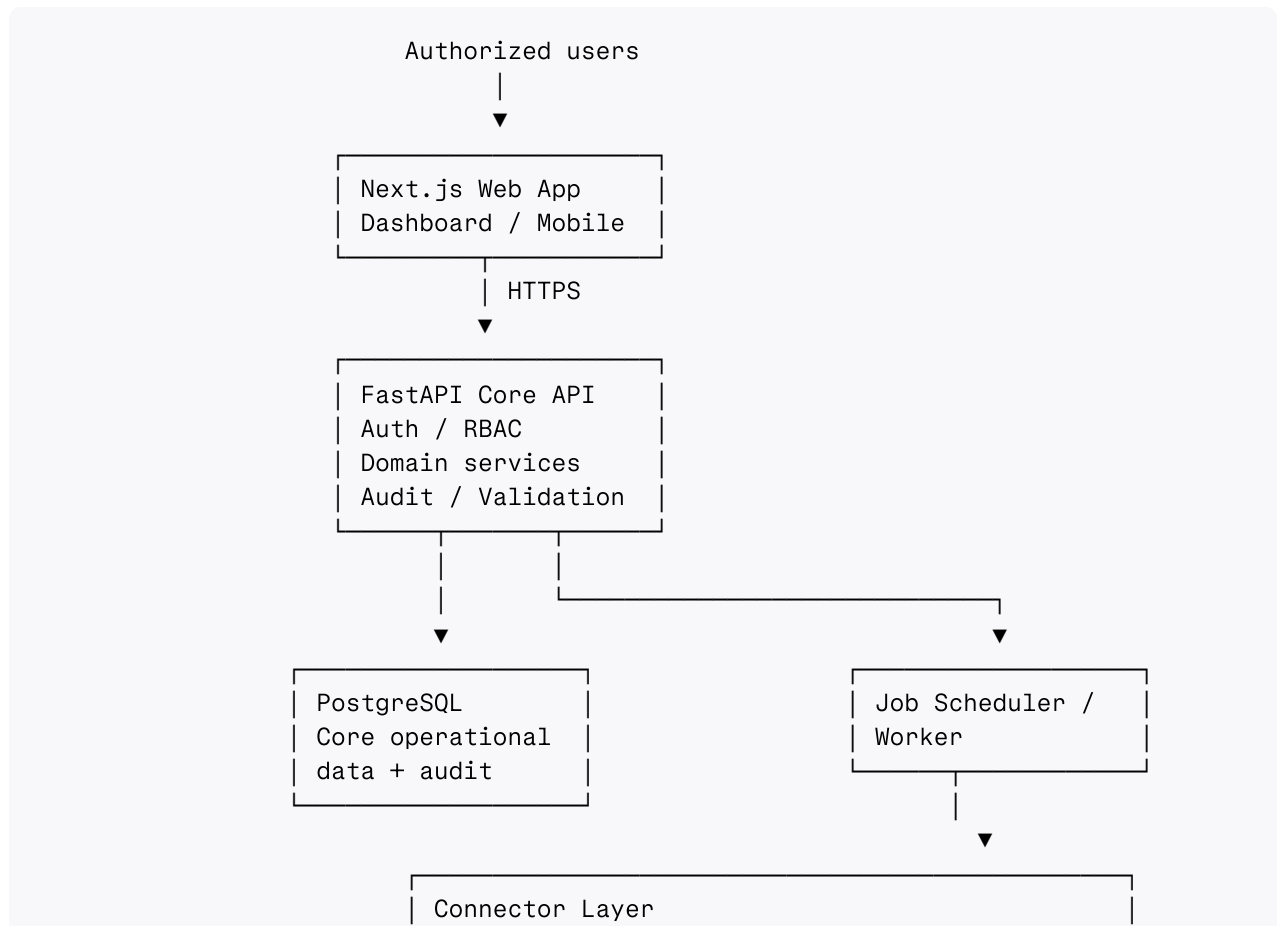
- Separate integration code from business/application logic.
- Make vendor-specific changes local to a connector module.
- Preserve a normalized, company-owned data model.
- Prevent browser clients from accessing vendor API secrets.
- Support multi-organization and multi-site authorization boundaries.
- Support scheduled data collection without blocking user requests.
- Preserve audit trails and data-freshness information.
- Start small using a modular monolith rather than prematurely adopting microservices.

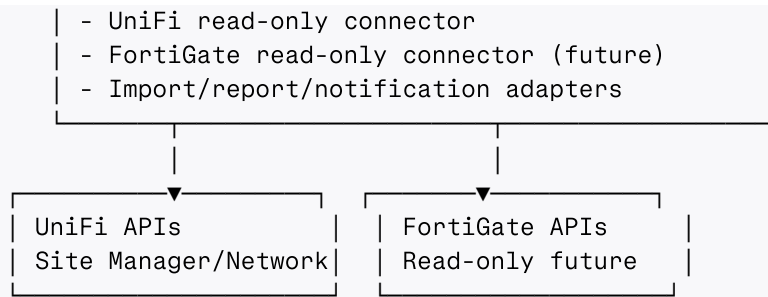
11. Recommended Technology Stack

Layer	Initial recommendation	Rationale
Web frontend	Next.js + TypeScript	Productive responsive internal dashboard and type-safe UI development
Backend API	Python + FastAPI	Strong API validation, OpenAPI generation, integration tooling, and automation ecosystem
Database	PostgreSQL	Relational integrity, multi-user access, reporting, JSON support, and strong auditing/query capabilities

Layer	Initial recommendation	Rationale
ORM/migrations	SQLAlchemy + Alembic	Explicit domain persistence and repeatable schema migration
Background scheduling	APScheduler or cron initially	Simple, understandable scheduled synchronization
Background queues later	Celery + Redis only when needed	Useful for increased concurrency, retry workflows, and long-running tasks
Integrations	Python connector modules	Good HTTP, SDK, parsing, and automation ecosystem
Deployment	Docker Compose on a secured Linux VM	Reproducible and operationally manageable initial deployment
Reverse proxy	Nginx, Caddy, or managed equivalent	TLS termination, routing, security headers, rate limiting as appropriate
Metrics	PostgreSQL initially; Prometheus/Grafana as needs grow	Avoid excessive operational complexity at MVP stage
Logs	Structured application logs; Loki/centralized logging later	Trace incidents/integrations without exposing secrets
Source control/CI	Git + CI pipeline	Repeatable builds, linting, tests, scans, and deployments
Infrastructure automation	Ansible; Terraform where provider support and project scope justify it	Controlled, repeatable infrastructure changes

12. Architecture Overview





13. Modular Monolith Structure

Start with one deployable backend composed of clear modules. This avoids distributed-system complexity while keeping future separation possible.

```
backend/  
  app/  
    api/  
      v1/  
        routes/  
        schemas/  
    core/  
      config.py  
      security.py  
      logging.py  
    domain/  
      organizations/  
      sites/  
      assets/  
      incidents/  
      knowledge_base/  
      changes/  
      reporting/  
    integrations/  
      unifi/  
      fortigate/  
      common/  
    jobs/  
      sync_unifi.py  
      reports.py  
      alerts.py  
    persistence/  
      models/  
      repositories/  
      migrations/  
    services/  
      reconciliation.py  
      authorization.py  
      auditing.py  
      health.py  
    tests/
```

Boundary rules

- UI code does not call UniFi or FortiGate directly.
- Vendor connectors do not contain UI rules or tenant-authorization decisions.
- Domain services decide what an imported device means to Blue Hawk Ops.
- Repository/persistence code handles database access only.
- API routes validate requests and invoke domain services; they do not embed complex business logic.
- Jobs invoke connector/domain services and create auditable results.

14. Data Model

14.1 Core entities

```
Organization 1 — N Site
Site         1 — N Location
Site         1 — N Asset
Asset        0 — 1 NetworkDevice
Site         1 — N Network
Site         1 — N Incident
Asset        1 — N AssetHistory
NetworkDevice 1 — N DeviceObservation
Integration  1 — N IntegrationRun
Incident     N — N KnowledgeArticle
Incident     N — N Alert
ChangeRequest N — N Asset / NetworkDevice
User         1 — N AuditEvent
```

14.2 Suggested database tables

Table	Purpose	Important fields
organizations	Client/business entities	id,name,status,contact_metadata
sites	Physical/logical locations	id,organization_id,name,timezone,status,notes
locations	Hierarchical physical placement	id,site_id,parent_id,type,name,description
assets	Company/client equipment inventory	id,site_id,location_id,asset_code,category,vendor,model,serial,mac,status
network_devices	Network-vendor identity and current state	id,asset_id,platform,external_id,controller_id,management_ip,firmware,last_seen_at
device_observations	Time-stamped imported state	id,network_device_id,observed_at,connectivity,metrics_json,source_run_id

Table	Purpose	Important fields
networks	VLAN/subnet records	id, site_id, vlan_id, name, cidr, purpose, risk_level
incidents	Support lifecycle records	id, site_id, priority, status, title, symptoms, resolution, verified_at
incident_events	Chronological incident updates	id, incident_id, actor_id, event_type, details, created_at
knowledge_articles	Runbooks/known-error documentation	id, scope, title, content, status, owner_id, review_due_at
alerts	Normalized operational conditions	id, site_id, device_id, source, fingerprint, severity, status, opened_at, closed_at
integrations	Integration configuration metadata	id, type, organization_id, site_scope, status, secret_reference
integration_runs	Import/export execution record	id, integration_id, started_at, finished_at, status, records_processed, error_summary
asset_discrepancies	Reconciliation review queue	id, asset_id, network_device_id, type, status, detected_at, resolution_note
change_requests	Planned operational changes	id, site_id, risk, plan, rollback_plan, approval_status, schedule
audit_events	Security/accountability evidence	id, actor_id, organization_id, action, entity_type, entity_id, before_json, after_json, created_at

14.3 Source and freshness fields

Any data imported from a vendor must carry its origin. At minimum, preserve:

```

source_platform      # unifi, fortigate, manual, csv_import, script
source_external_id   # Vendor-side stable ID when available
source_run_id        # Integration run that produced the value
observed_at          # Time the vendor state was observed
synced_at            # Time Blue Hawk Ops stored the value
freshness_status     # fresh, delayed, stale, unknown

```

This prevents a key operational failure: showing an old API value as if it is live truth.

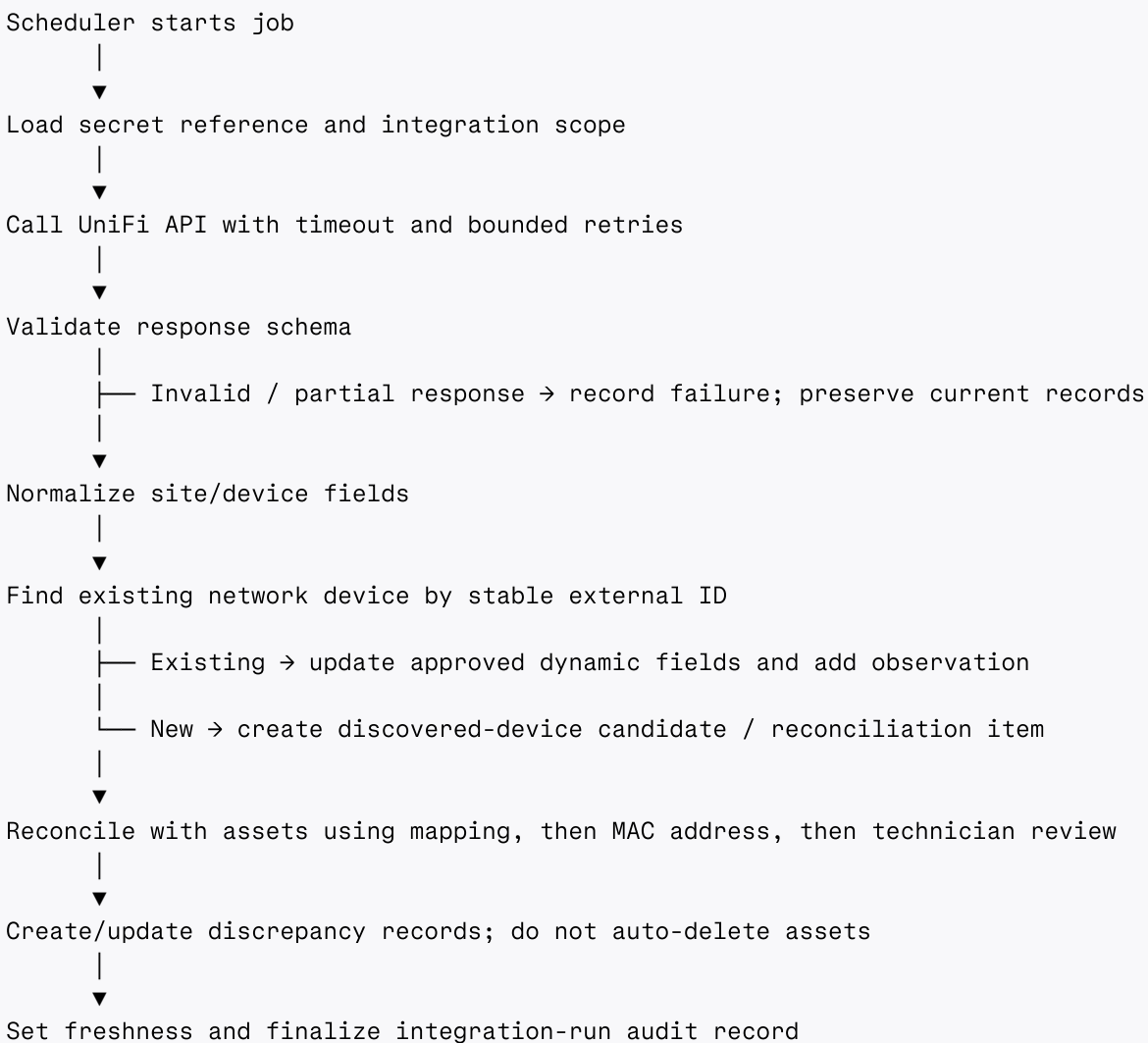
15. UniFi Connector Design

15.1 Responsibilities

The UniFi connector should:

- Authenticate securely using a dedicated least-privilege credential.
- Discover approved organization/site scope.
- Fetch allowed site and device data on a schedule.
- Handle pagination, errors, timeouts, vendor rate limits, and partial failures.
- Validate external payloads before persistence.
- Map vendor-specific fields into normalized internal records.
- Preserve stable external identifiers and source metadata.
- Produce a clear integration-run result.
- Never expose vendor credentials to the browser.
- Remain read-only in initial phases.

15.2 Synchronization sequence



15.3 Matching rules

Use a conservative hierarchy:

1. Existing approved mapping between `network_device.external_id` and an asset.
2. Exact normalized MAC address match.
3. Exact serial/model match only if reliably available and confirmed.
4. Manual technician review.

Never rely solely on display name or IP address; both can change or be reused.

15.4 Dynamic versus authoritative fields

Field	Initial source of truth	Automation behavior
Asset code	Blue Hawk Inventory	Never overwrite from UniFi
Physical location	Blue Hawk technician/inventory	Flag differences; do not auto-overwrite
Owner/responsible person	Blue Hawk Inventory	Never overwrite from UniFi
Purchase/warranty information	Blue Hawk Inventory	Never overwrite from UniFi
Device external ID	UniFi mapping	Store and preserve after approval
MAC address	UniFi + verified inventory	Flag conflicts for review
Model/device type	UniFi discovery	Import/update with source metadata
Firmware version	UniFi observation	Update as time-stamped dynamic data
Online/offline / last seen	UniFi observation	Update as time-stamped dynamic data
Client count / health metric	UniFi observation	Store as observation/metric, not permanent asset truth

16. FortiGate Connector Design — Later

FortiGate integration should be a separate module and later milestone because firewall/VPN/security data is more sensitive and incorrect access can have greater operational impact.

Initial approved read-only collection may include:

- Appliance identity, serial, model, FortiOS version.
- WAN/interface state.
- VPN status.
- High-level policy metadata, carefully role restricted.
- Backup status and last verification date.
- Selected health/event counters.
- Administrative/security review reminders.

Do not automate policy writes or configuration changes in the initial connector. Any future write workflow requires a dedicated change-management design, lab testing, review, approval, maintenance window, snapshots/backups, post-change validation, rollback plan, and complete audit history.

17. Internal REST API

Blue Hawk Ops exposes its own stable, business-oriented API. It should not expose raw vendor endpoints directly to users.

Example endpoints

```
GET    /api/v1/organizations
GET    /api/v1/organizations/{organization_id}/sites
GET    /api/v1/sites
GET    /api/v1/sites/{site_id}/health
GET    /api/v1/sites/{site_id}/devices
GET    /api/v1/assets
POST   /api/v1/assets
GET    /api/v1/assets/{asset_id}
PATCH /api/v1/assets/{asset_id}
GET    /api/v1/discrepancies
POST   /api/v1/discrepancies/{discrepancy_id}/resolve
GET    /api/v1/incidents
POST   /api/v1/incidents
PATCH /api/v1/incidents/{incident_id}
GET    /api/v1/knowledge-articles
POST   /api/v1/knowledge-articles
GET    /api/v1/integrations/runs
POST   /api/v1/integrations/{integration_id}/sync
GET    /api/v1/reports/site-summary
POST   /api/v1/change-requests
```

API rules

- Version routes from the beginning, e.g., /api/v1.
- Apply tenant/organization authorization to every query.
- Paginate list endpoints.
- Use consistent error objects and do not reveal tokens, stack traces, internal addresses, or vendor-sensitive details to standard users.
- Validate request bodies with typed schemas.
- Document the API with OpenAPI.
- Require audit events for material mutations and exports.
- Restrict manual synchronization endpoints to approved roles and rate-limit them.

18. Security Architecture

18.1 Access model

Client viewer

→ Own organization only, read-only reports/incidents explicitly shared

Support technician

→ Authorized organizations/sites, assets/incidents/runbooks; no integration secrets

Network operator

→ Network operational view, incident workflow, change-request creation

Platform administrator

→ Users, roles, integration configuration metadata, system settings

Vendor integration worker

→ Dedicated non-human identity with only approved read scopes

18.2 Secrets model

- Store vendor credentials outside the frontend and source repository.
- Prefer a secrets manager when available; otherwise use protected server environment variables with restricted deployment access.
- Record only a secret reference or integration identifier in the database, never the raw secret.
- Rotate credentials on a defined schedule and immediately upon suspected exposure.
- Use separate credentials per integration/client scope where feasible.
- Do not reuse a technician's administrative personal account as an application integration credential.

18.3 Audit model

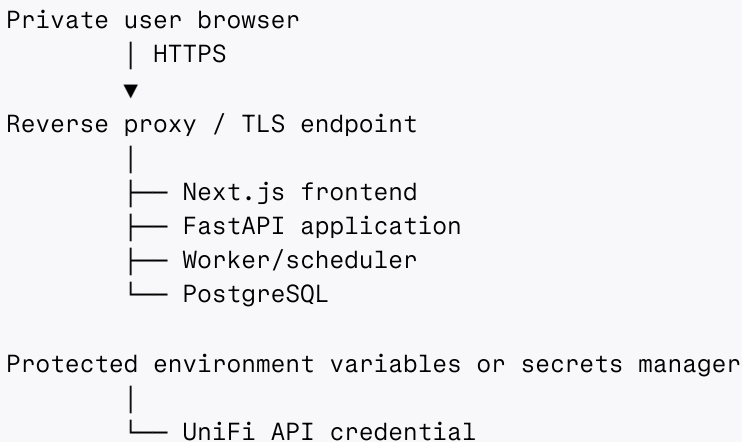
Audit these events:

- Login success/failure and session-related security events.
- User/role/permission changes.
- Integration creation, scope changes, disablement, and manual sync requests.
- Asset creation, retirement, ownership/location updates, and import actions.
- Incident status/priority/assignment changes.
- Data exports and report generation.
- Change-request approval/implementation/closure.
- Every future action that can affect an external system.

Do not write passwords, tokens, private keys, session cookies, full sensitive raw payloads, or unredacted confidential data to audit logs.

19. Deployment Model

MVP deployment



Deploy the MVP on a secured Linux VM, private cloud environment, or approved internal server. Use Docker Compose to make the deployment reproducible. Keep PostgreSQL data persistent and backed up. Restrict administrative access through VPN or an approved protected path where possible.

Backup and recovery minimums

- Database backup on a defined schedule.
- Encrypted backup storage appropriate to company policy.
- Separate backup copy outside the primary host where feasible.
- Deployment/configuration documentation.
- Restore test performed at a defined interval.
- Configuration backup references for vendor systems are tracked, but vendor configuration backups remain separately secured.

Part V — Product Roadmap

20. Delivery Phases

Phase 0 — Discovery and Approval

Goal: ensure the project solves a real approved operational problem without gathering more access/data than necessary.

Deliverables:

- Stakeholder interviews: technicians, manager, network administrator, and pilot-client owner as appropriate.
- Inventory of existing tools, data sources, tickets, access controls, and pain points.
- Data classification and client authorization review.
- One-page product brief.
- Threat model.
- Architecture decision record for modular monolith, PostgreSQL, FastAPI, and UniFi read-only integration.
- Pilot success criteria and rollback plan.

Phase 1 — Foundation and Manual Operations MVP

Goal: create a useful internal product before any vendor integration.

Features:

- Authentication and RBAC.
- Organizations, sites, physical locations.
- Manual assets and network device records.
- Search, filters, and audit history.
- Basic incidents and knowledge articles.
- API health checks, structured logs, PostgreSQL backups, and Docker Compose deployment.

Exit criteria: a technician can record an asset, find it by serial/MAC/code, create an incident, document a resolution, and access data only within their authorized scope.

Phase 2 — UniFi Read-Only Pilot

Goal: prove secure, useful device discovery and status synchronization for one approved low-risk pilot site.

Features:

- Dedicated least-privilege UniFi credential.
- Scheduled sync of approved site/device data.
- Integration-run logs and freshness indicators.
- Device discovery and reconciliation queue.
- Device details: model, MAC, management IP, firmware, status, last seen where available.

- A basic site health dashboard.

Exit criteria: data synchronizes correctly; secrets remain protected; unmatched/mismatched assets are reviewable; no production configuration change is performed.

Phase 3 — Incidents, Alerts, and Reports

Goal: turn visibility into operational value.

Features:

- Alert deduplication and repeated-check confirmation.
- Link device health conditions to incidents.
- Guided incident templates and runbook links.
- Site/device summary reports.
- Inventory discrepancy report.
- Data freshness and failed-sync report.

Exit criteria: the pilot generates a useful weekly report and supports a real troubleshooting workflow with evidence and closure verification.

Phase 4 — Multi-Site UniFi Operations

Goal: scale the proven model across approved client sites with strict tenant boundaries.

Features:

- Multi-organization/site permissions.
- Per-site integration scope/credentials.
- Cross-site internal operations view for approved employees.
- Client-restricted reports.
- Maintenance reminders and firmware-review tracking.
- Configuration-backup tracking.

Phase 5 — FortiGate Read-Only Visibility

Goal: add security/perimeter context without allowing configuration writes.

Features:

- FortiGate asset/identity records.
- Interface/WAN/VPN status.
- Selected backup and firmware review information.
- Strictly restricted policy/security summaries.
- Change-reference and compliance-review reminders.

Phase 6 — Controlled Automation, Only if Approved

Goal: enable narrowly scoped, reversible, reviewable actions.

Preconditions:

- Stable data model and synchronization.
- Least-privilege roles and MFA.
- Documented change-management process.
- Auditing and alerting.
- Backups and tested restoration.
- Lab or low-risk validation.
- Explicit management approval.

Possible actions:

- Generate scheduled reports.
- Create tickets/notifications from confirmed health conditions.
- Request approved maintenance windows.
- Trigger narrowly scoped vendor actions only after explicit human confirmation and post-action validation.

21. Initial Product Backlog

Must build first

- [] Product brief and pilot scope.
- [] Threat model and data classification notes.
- [] Git repository, issue tracker, coding standards, `.env.example`, and secrets policy.
- [] PostgreSQL schema and Alembic migrations.
- [] FastAPI service, OpenAPI docs, structured logs, health endpoint.
- [] Next.js dashboard shell.
- [] Login, RBAC, organization/site filtering.
- [] Manual assets, locations, and network-device records.
- [] Incident creation and activity timeline.
- [] Knowledge-base/runbook records.
- [] Audit events.
- [] Docker Compose and tested database-backup process.

UniFi pilot

- [] Dedicated integration identity/API key with approved read scope.
- [] UniFi connector module with mocked tests.
- [] Integration run record and health status.
- [] Site/device sync for one pilot.
- [] Data freshness display.
- [] Asset/device reconciliation queue.
- [] Device details page.
- [] Manual sync restricted to administrators.

After successful pilot

- [] Alert confirmation and deduplication.
- [] Incident creation from an alert.
- [] Weekly client/site report.
- [] QR asset lookup.
- [] Attachment handling/security controls.
- [] Second-site rollout.
- [] FortiGate connector design review.

Part VI — Operations, Testing, and Risk

22. Test Strategy

Unit tests

- Domain rules: asset status transitions, tenant checks, role checks, discrepancy logic, alert fingerprinting.
- Connector field mapping and malformed vendor payload handling.
- Data-normalization functions: MAC address normalization, timestamps, IDs, device types.
- API schema validation and error handling.

Integration tests

- PostgreSQL migrations and repository behavior.
- Authentication/RBAC and tenant isolation.
- API routes with test database.
- Mocked UniFi API success, timeout, pagination, partial error, invalid payload, and rate-limit scenarios.

- Scheduler/job behavior and retry/backoff limits.

Security tests

- Authorization tests for every organization/site-scoped endpoint.
- Attempted cross-tenant record access.
- Secret-leak scanning in repository/CI.
- Dependency vulnerability scanning.
- File-upload validation if attachments are enabled.
- Audit-log coverage for privileged operations.

User acceptance tests

- Technician locates an asset by serial, MAC, or code.
- Technician creates and closes a Wi-Fi incident with verification notes.
- Admin links a discovered UniFi device to an asset.
- Manager views an authorized site report.
- Client viewer cannot see another client's site, asset, incident, topology, or report.
- Failed integration shows stale-data warning and preserves existing records.

23. Risk Register

Risk	Likelihood	Impact	Mitigation
Scope becomes a replacement for every vendor platform	High	High	Maintain explicit out-of-scope list; deliver small pilot milestones
API credentials exposed	Medium	Critical	Secrets manager/environment controls, scanning, rotation, never expose to browser
Cross-client data exposure	Medium	Critical	Server-side tenant filtering, RBAC tests, least privilege, audit logs
Stale data treated as live truth	Medium	High	Freshness timestamps/status, integration-run visibility, failed-sync alerts
Wrong asset matching	Medium	Medium	Stable IDs/MAC-first matching, review queue, no automatic destructive reconciliation
Vendor API changes	Medium	Medium	Isolate connectors, schema validation, contract tests, graceful degradation
Automation causes outage	Low initially	Critical	Read-only first; change control, confirmation, lab validation, rollback requirements
Excessive telemetry/cost/complexity	Medium	Medium	Start with low-frequency health/inventory data; define retention before scale
Incomplete documentation	High	Medium	Make docs/runbooks/ADRs part of definition of done

Risk	Likelihood	Impact	Mitigation
Project loses stakeholder support	Medium	High	Pilot one visible pain point; show time saved and real operational outcomes

24. Definition of Done

A Blue Hawk Ops feature is complete only when:

- The user problem and scope are documented.
- Functional and security requirements are identified.
- Authorization is implemented and tested.
- Input validation and error handling are present.
- Audit behavior is defined for material actions.
- Automated tests cover core behavior and failure paths.
- User-facing documentation or a runbook exists where needed.
- Monitoring/logging gives enough evidence to troubleshoot the feature.
- Secrets are not committed or exposed.
- Deployment and rollback steps are documented.
- The feature was reviewed against tenant isolation and least-privilege requirements.

Part VII — Relationship to BlueHawk Inventory and IT Toolkit

25. Product Boundaries

Blue Hawk Ops should not immediately replace BlueHawk Inventory or IT Toolkit. They can share concepts and data while retaining clear primary purposes.

Product	Primary purpose	Integration relationship
BlueHawk Inventory	Asset lifecycle, ownership, physical location, maintenance, photos, QR, and inventory verification	Provides/owns asset records that UniFi-discovered devices can reconcile against
IT Toolkit	Technician diagnostics, guided troubleshooting, local system/network checks, learning, and support reporting	Consumes approved Ops context for the selected site/device/asset and links results to incidents
Blue Hawk Ops	Multi-site operational visibility, integrations, incidents, reports, audit, workflow, and data correlation	Shared operations layer that consumes vendor data and relates it to inventory and support work

Recommended evolution

1. Build or validate the manual asset/inventory foundation first.
2. Add a **shared internal data model or API**, not separate direct UniFi integrations in each UI.
3. Use BlueHawk Inventory as the source of truth for physical/lifecycle fields.
4. Use Blue Hawk Ops for scheduled vendor data, health, multi-site views, discrepancies, alerts, incidents, and reports.
5. Let IT Toolkit query the internal API for safe troubleshooting context, not vendor APIs directly.

This avoids duplicate secrets, duplicate data transformations, inconsistent device records, and multiple apps competing to decide what is true.

Part VIII — Approval Questions Before Development

26. Decisions to Confirm

Before beginning a real pilot, answer these questions with the responsible company personnel:

1. Which exact site will be the pilot, and who owns approval for it?
2. What business problem is highest priority: inventory accuracy, offline-device visibility, incident documentation, reporting, or another issue?
3. Which UniFi API access model and permissions are approved?
4. Is the information for one organization only, or multiple client organizations from the beginning?
5. What data may be stored centrally, and what must remain only in native vendor consoles?
6. Who can see client topology, device identity, alerts, logs, and reports?
7. Where will the application run: approved internal VM, private cloud, VPS, or another managed environment?
8. What identity/authentication system should users use?
9. What are the backup, data-retention, and incident-response requirements?
10. Is a client-facing portal needed now, or only an internal dashboard?
11. Is there an existing ticketing system that should be linked instead of replaced?
12. Which actions, if any, could ever be automated—and what approval and rollback rules would apply?

Part IX — Initial Pitch

27. Pilot Proposal

Proposal: Build a read-only internal operations dashboard for one approved Blue Hawk client site. The dashboard will synchronize authorized UniFi device information, reconcile it with the asset inventory, show site/device status and data freshness, and link device events to incidents and documented resolutions. It will not modify UniFi, FortiGate, or other production configurations.

Pilot outcomes

- Faster identification of affected site/device during support incidents.
- Fewer gaps between discovered network hardware and the recorded inventory.
- Consistent incident notes and reusable troubleshooting documentation.
- A simple weekly operational report.
- A secure, auditable technical foundation for later approved integrations.

Pilot success criteria

- At least one approved site synchronizes reliably on a defined schedule.
- Authorized users can find a device by name, MAC, model, or asset code.
- The system clearly shows when imported data was last refreshed.
- Unmatched/mismatched assets appear in a review queue.
- A technician can create, document, verify, and close a linked incident.
- Access is tenant-restricted and role-based.
- No vendor configuration writes occur.
- The project produces a short operations report demonstrating value, limitations, and next steps.

Appendix A — Incident Note Template

```
# Incident INC-YYYY-####: [Short title]
```

```
## Scope and impact
```

- Organization/site:
- Affected users/assets/services:
- Priority:
- Start time:

```
## Symptoms and evidence
```

- Reported symptoms:
- Observed evidence:
- Relevant device/site data:

```
- Recent changes:

## Diagnostic timeline
1. Observation:
2. Hypothesis:
3. Test:
4. Result:
5. Next step:

## Resolution
- Corrective action:
- Configuration/change reference:

## Verification
- Test performed:
- Result:
- User/client confirmation:

## Prevention / follow-up
- Root cause:
- Preventive action:
- Linked runbook/knowledge article:
```

Appendix B — Change Request Template

```
# Change CHG-YYYY-####: [Short title]

## Objective

## Affected organization/site/assets

## Business impact and risk

## Preconditions

## Implementation plan

## Validation plan

## Rollback plan

## Maintenance window

## Approval
- Requested by:
- Approved by:
- Date/time:

## Execution results

## Closure and evidence
```

Appendix C — Architecture Decision Record Template

```
# ADR-###: [Decision]

## Status
Proposed | Accepted | Superseded

## Context
Describe the problem, constraints, security considerations, stakeholders, and operational requirements.

## Decision
State the chosen approach.

## Alternatives considered
| Alternative | Benefits | Costs/Risks | Why not selected |
|---|---|---|---|

## Consequences
Describe security, cost, reliability, performance, maintenance, and training consequences.

## Validation
State how the decision will be tested or reviewed.
```